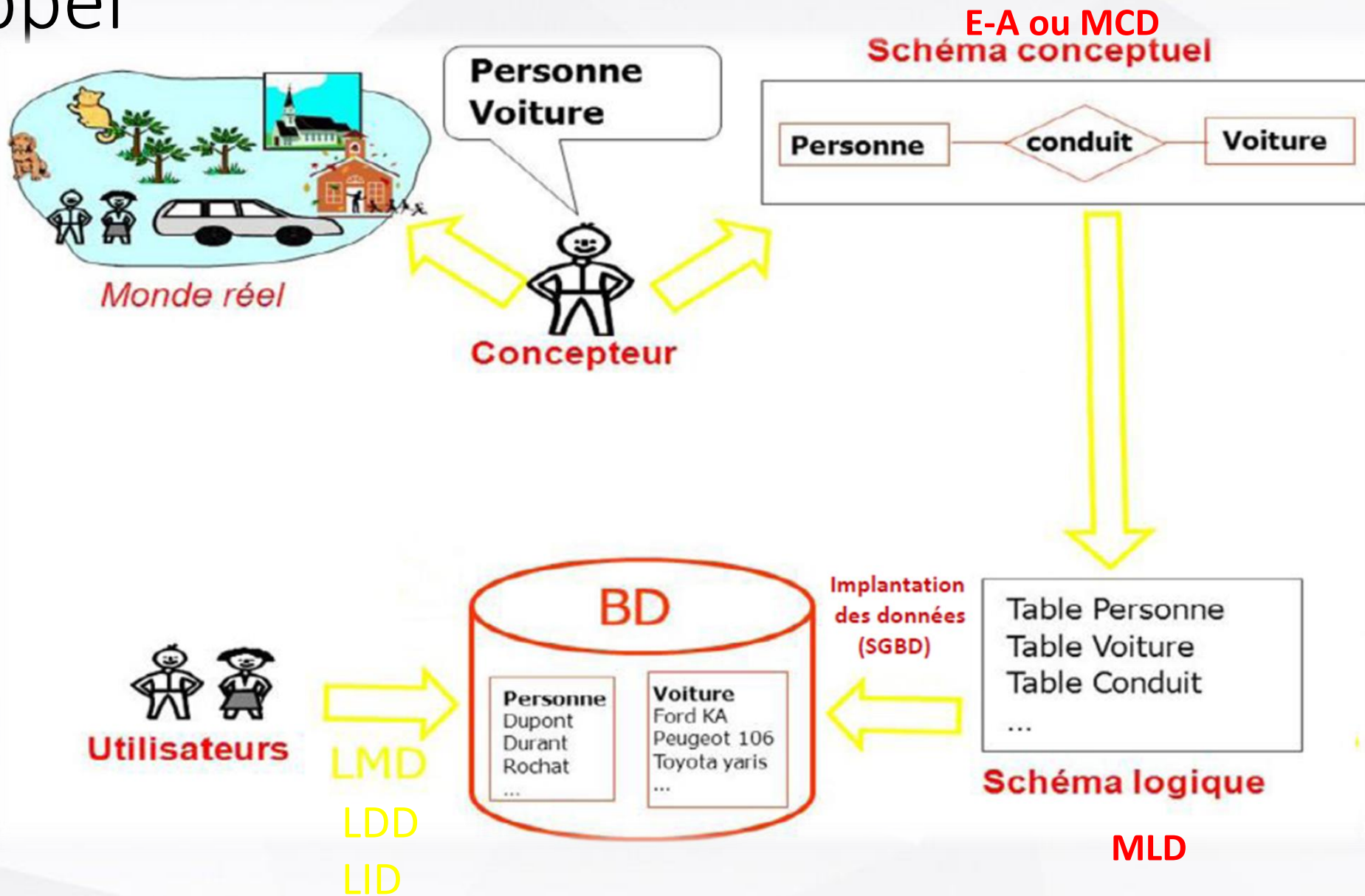


Module 24 : Système d'information et base de données

Chapitre 4 : Langage SQL (Partie 1)

1. Introduction au langage SQL
2. Langage de définition de Données (LDD)
3. Langage de Manipulation des Données (LMD)
4. Langage d'Interrogation de Données de Données (LID)

Rappel



1

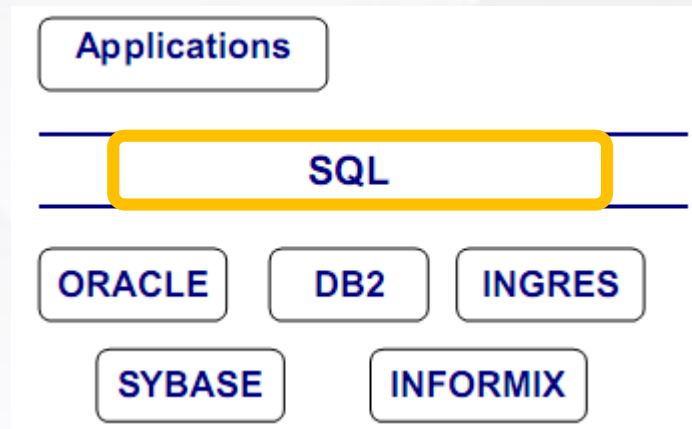
■ Introduction au langage SQL

Introduction

- **SQL : Structured Query Language** (Langage de requêtes structuré).
- C'est un langage complet de gestion des BD relationnelles conçu par IBM (années 70).
- SQL est un langage de **définition de données (LDD)**, de **manipulation de données (LMD)**, et de **contrôle de données (LCD)** pour les **bases de données relationnelles**.
- SQL est devenu le langage standard des **SGBD relationnelles**.
 - MySQL
 - DB2, Oracle
 - Informix, Ingres
 - RDB, ...
- Chacun de ces SGBDR a cependant sa propre **variante du langage**.



- **SQL** est l'**interface logiciel/logiciel** entre les applications et les BDR.



- Les **commandes** du langage **SQL** peuvent :
 - être tapées directement au clavier par l'utilisateur.
 - être incluses dans un programme écrit dans un langage de programmation grâce à un pré-compilateur ou une librairie d'accès au SGBD.

Historique

- **1970** : E.F. **CODD** invente le **modèle relationnel** qui repose sur l'**algèbre relationnelle**.
- **1977** : Création du langage **SEQUEL** (Structured English Query Language).

SEQUEL continue de s'enrichir pour devenir **SQL**.

- **1981** : La société **ORACLE** CORP lance la première version de son **SGBD relationnelle**.
- **1986** : La société **SYBASE** lance son **SGBDR** conçu selon le modèle **Client-Serveur**.

La première norme **SQL (SQL-1)** de l'ISO apparaît.

- **Depuis ...**

Les différents produits ont évolué, la norme **SQL1** est passée à **SQL-2**, puis **SQL-3**.

SQL est désormais un langage incontournable pour tout SGBD moderne.

Catégories d'instruction SQL

- Les instructions SQL sont regroupées en catégories en fonction de leur utilité et des entités manipulées.
- SQL est à la fois un :
 - **Langage de Définition des Données (LDD)** : **CREATE**, **ALTER**, **DROP**
 - définition des éléments d'une base de données : tables, colonnes, clefs, index, contraintes, ...
 - **Langage de Manipulation de Données (LMD)** : **UPDATE**, **INSERT**, **DELETE**
 - manipulation des données stockées dans la BD : insertion, suppression, modification, extraction, ...
 - **Langage d'Interrogation de Données (LID)** : **SELECT**
 - **Langage de Contrôle des Données et des utilisateurs (LCD)** : **GRANT**, **REVOKE**.
 - gestion des droits d'accès aux données : acquisition et révocation des droits.

Normes SQL

- SQL a été **normalisé** dès **1986** mais les premières normes, trop incomplètes, ont été ignorées par les éditeurs de SGBD.
- SQL définit plusieurs niveaux de normalisation :
 - **SQL1** : norme de base.
 - **SQL2** : appelée aussi **SQL92** (apparue en 1992) , c'est une extension de **SQL1**.
 - *f* meilleur support des règles du **relationnel**.
 - *f* types de données plus variés.
 - **SQL3** : intégration du **modèle objet**.

Langage SQL

Caractéristiques

- SQL est supporté par la plupart des SGBD : que ce soit les SGBD **micro** tel que **Access** ou les produits plus **professionnels** tels que **Oracle ,MySQL etc...**
- Son succès est dû essentiellement à sa **simplicité** et au fait qu'il s'appuie sur le **schéma conceptuel** pour énoncer des requêtes.
- Il propose un **langage relationnel** de **requêtes ensemblistes** pour manipuler les tables .
- SQL ne possède pas la **puissance** d'un **langage de programmation** :
 - entrées/sorties
 - instructions conditionnelles
 - Boucles
 - affectations, ...
- Pour certains traitements il est nécessaire de **coupler** le langage **SQL** avec un **langage de programmation** plus complet.

1

▪ Langage de Définition des Données (LDD)

Définition

- Le **LDD** est un langage orienté au niveau de la **structure** de la **BD**.
- Le LDD permet de :
 - créer, modifier, supprimer des objets.
 - définir le domaine des données (nombre, chaîne de caractères, date, booléen, ...)
 - ajouter des contraintes de valeur sur les données.
 - autoriser ou d'interdire l'accès aux données.
- Les instructions du LDD sont :
 - CREATE
 - ALTER
 - DROP ...

Créer une table

- Lors de la création d'une table, la requête SQL doit prendre en considération les **Contraintes d'Intégrité (CI)** précisées lors de la création de la **BD** :
 - **CI de domaine :**
 - préciser, en plus du **nom**, le **type** de chaque **colonne (attribut)**.
 - **CI de table (ou de relation ou d'entité) :**
 - Une clé primaire doit toujours être unique dans une table. Cette contrainte est appelée **contrainte d'intégrité de relation**.
 - Si un attribut est inconnu ou non défini, on introduit valeur nulle (NULL).
 - Une clé primaire ne peut avoir une valeur nulle.
 - **CI de référence :**
 - Lors de l'insertion/suppression de tuples, il faut prendre en considération les **contraintes d'intégrité référentielle**.

Créer une table: CREATE TABLE

- Une table est un ensemble de **lignes** et de **colonnes**.
- La création consiste à définir (en fonction de l'analyse) :
 - Le **nom** de chaque colonne;
 - Le **format** (*type*) de chaque colonne;
 - La **valeur** par défaut à la création de la ligne (DEFAULT);
 - Les **règles de gestion** s'appliquant à la colonne (CONSTRAINT)

Création simple d'une table

❖ Création simple : **CREATE TABLE**

- La commande de création de table la plus simple ne comportera que le **nom** et le **type** de chaque **colonne** de la table.
- A la création, la table sera vide, mais un certain espace lui sera alloué.

Syntaxe :

```
CREATE TABLE nom_table (nom_col1 TYPE1, nom_col2 TYPE2, ...);
```

- Quand on crée une table, il faut définir les **contraintes d'intégrité** que devront respecter les données que l'on mettra dans la table.

Remarque : pour créer une base de données on utilise le syntaxe : **CREATE DATABASE** nom_database ;

Langage de Définition des Données

Création simple d'une table

▪ Types alphanumériques

- **CHAR** Un caractère
- **CHAR(n)** Chaîne de n caractères. (1 par défaut)
- **VARCHAR(n), VARCHAR2(n)** Chaîne dynamique d'au plus n caractères
- **Text, LONG** Chaîne pouvant atteindre jusqu'à 65000 caractères.

▪ Types numériques

- **INT*, INTEGER** Nombre entier signé
- **SMALLINT, INT2*** entier codé sur 2 octets
- **MEDIUMINT, INT4*** entier 4 octets
- **BIGINT, INT8*** entier 8 octets
- **NUMBER[(n[,d])]** Nombre de n chiffres dont éventuellement d après la virgule.
- **REAL, FLOAT, FLOAT4*** Réels 4 octets
- **DOUBLE PRECISION, FLOAT8*** Réels 8 octets
- **NUMERIC[(m,e)], DECIMAL[(m [,e])]** nombre réel de mantisse à m chiffres et d'exposant e
- **RAW (n)** Binaire de longueur n (pour stocker des images par exemple).

Création simple d'une table

▪ Autres types

- **DATE** structure composée de champs et stockant le jour, mois, année
- **TIME** structure composée de champs et stockant l'heure, les min et les sec.
- **TIMESTAMP** permet de stocker des données constituées d'une date et d'une heure
- **INTERVAL*** intervalle de dates
- **ROWID** Pointeur de ligne
- **MONEY** permet de stocker des valeurs monétaires.
- **BOOLEAN** permet de stocker des valeurs Booléenne.

▪ Remarque

- Indépendamment des types, la valeur **NULL** est une valeur spéciale qui dénote l'absence de valeurs.
- Un entier, une chaîne, une date peuvent avoir pour valeur **NULL**.

Création simple d'une table

- *Exemple 1:*
 - Créer la table **EMP** (numemp, nom, dateNaiss, salaire, job)

```
CREATE TABLE EMP(  
    numemp number,  
    nom varchar2(30),  
    dateNaiss date,  
    salaire number(10,2),  
    job varchar2(20)  
);
```

Langage de Définition des Données

Création simple d'une table

- *Remarque :*

- On peut donner une valeur par défaut pour une colonne en utilisant le mot clé **DEFAULT**.

- *Exemple 2:*

- créer la table **article** (**ref**, **nom**, **prix**, **datea**) en donnant à **datea** la valeur par défaut de date courante (**CURRENT DATE**) :

```
CREATE TABLE article (  
    ref VARCHAR(10),  
    nom VARCHAR(30),  
    prix DECIMAL(9,2),  
    datea DATE DEFAULT CURRENT_DATE);
```

Création avec insertion de données

- On peut insérer des données dans une table lors de sa création.
- *Syntaxe :*
 - **CREATE TABLE** **nom_table** [(nom_col1, nom_col2, ...)] **AS SELECT ...**
 - On peut ainsi, en un seul ordre SQL **créer** une table et la **remplir** avec des données provenant du résultat d'un **SELECT**.
- *Remarques :*
 - Si les **types des colonnes** ne sont pas spécifiés, ils correspondront à ceux du **SELECT**.
 - Si les **noms des colonnes** ne sont pas spécifiés, ils correspondront à ceux du **SELECT**.
 - Le **SELECT** peut contenir des fonctions de groupes mais pas d'**ORDER BY** car les lignes d'une table ne peuvent pas être classées.

Création avec insertion de données

Exemple :

- Créer la table **VENDEURS** (numvd, nom, dateNaiss, salaire) à partir de la table **EMP** (numemp, nom, dateNaiss, salaire, job).
- Requête :

```
CREATE TABLE VENDEURS(  
    numvd number,  
    nom char(20),  
    dateNaiss date,  
    salaire number(10,2)  
)  
AS SELECT * FROM EMP  
WHERE job='VENDEUR';
```

Création d'une table: Contraints d'intégrité

- Lors de la création d'une table, les types de contraintes d'intégrité (CI) à satisfaire sont :
 - CI d'*unicité*
 - CI de *référence*
 - CI de *domaine*
- *Syntaxe :*
 - La syntaxe générale d'une déclaration de contrainte est de la forme :

CONSTRAINT <nomContrainte> <typeContrainte>

Création d'une table: Contraints d'intégrité

▪ Contrainte **PRIMARY KEY** :

- Désigne l'attribut comme clé primaire de la table.
- Toute définition de table doit comporter au moins une contrainte de type **PRIMARY KEY**.
- La contrainte **PRIMARY KEY** est totalement équivalente à la contrainte **UNIQUE NOT NULL**.

▪ *Exemples :*

- Clé primaire non composée :

Create table T1(num number PRIMARY KEY, ...);

- Clé primaire composée :

Create Table T2(nump number, numt number, PRIMARY KEY(nump, numt), ...);

Création d'une table: Contraints d'intégrité

▪ *Remarque:*

- Il vaut mieux nommer toutes contraintes en utilisant la clause :

CONSTRAINT nomContrainte

de façon à pouvoir les gérer par la suite ou les retrouver facilement.

▪ *Exemple :*

- Créer la table **Article** (ref, nom, prix, datea) en indiquant que :
 - **ref** est la clé primaire de la table;
 - **nom** est unique.

CREATE TABLE article (

Ref VARCHAR(10) CONSTRAINT pk_article PRIMARY KEY,

nom VARCHAR(30) CONSTRAINT nom_unique UNIQUE,

prix DECIMAL(9,2),

datea DATE);

Création d'une table: Contraints de référence

▪ **Contrainte FOREIGN KEY :**

- Les **contraintes de référence** permettent d'assurer la liaison entre des attributs de 2 tables différentes.

▪ *Syntaxes :*

FOREIGN KEY (attribut, ...) REFERENCES table (attribut,...)

▪ *Exemple :*

CREATE TABLE Note (

numEtud NUMBER,

numMat NUMBER,

CONSTRAINT PK_dtvnt PRIMARY KEY (numEtud, numMat),CONSTRAINT FK_ref

FOREIGN KEY (numEtud) REFERENCES Etudiant(numEtudiant)

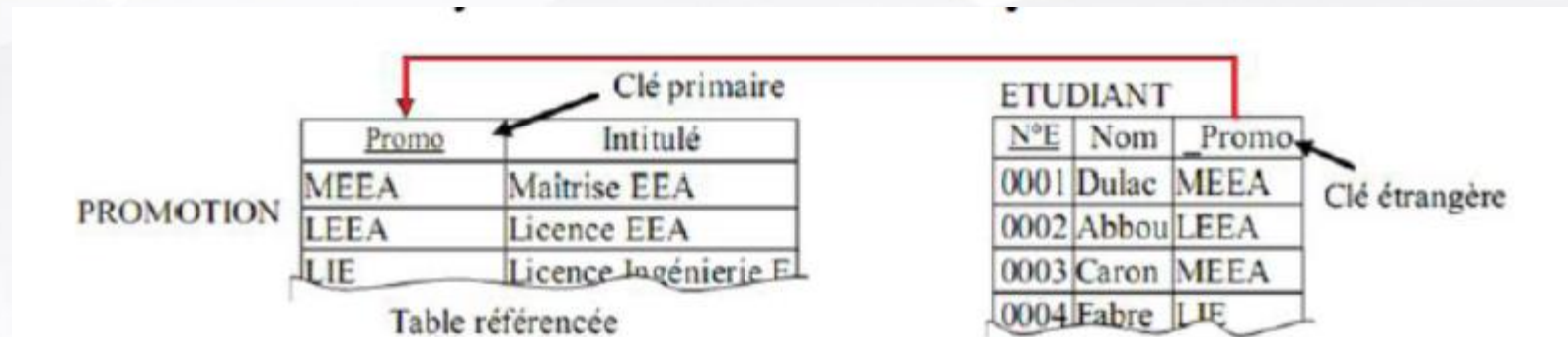
ON DELETE SET NULL ON UPDATE CASCADE

Langage de Définition des Données

Création d'une table: Contraints de référence

▪ Contrainte FOREIGN KEY :

- L'**intégrité référentielle** permet d'assurer la **cohérence** des données de liaison en **cas de mise à jour** ou de **suppression** d'une clé.
- A cet effet, on utilise, avec les actions référentielles (**ON DELETE** et **ON UPDATE**), les clauses additionnelles :
 - **CASCADE** : **DELETE** ou **UPDATE** effectuée sur la table référencée est lancée sur les attributs de même valeur de la table référençant.
 - **SET NULL** : donne à l'attribut référençant la valeur **NULL** lorsque l'on manipule la valeur correspondante de l'attribut référencé.
 - **SET DEFAULT** : impose une valeur par défaut à l'attribut référençant.
 - **NO ACTION** : inhibe le processeur d'intégrité. L'opération menée sur la table référencée est sans objet sur la table référençant.



Création d'une table: Contraints de référence

▪ Exemple :

```
CREATE TABLE DetailVentes (  
    venteId NUMBER,  
    numPiece NUMBER,  
    CONSTRAINT PK_dtvnt PRIMARY KEY (venteId, numPiece),  
    CONSTRAINT FK_ref1 FOREIGN KEY (venteId) REFERENCES vente(venteId)  
    ON DELETE SET NULL ON UPDATE CASCADE,  
    CONSTRAINT FK_ref2 FOREIGN KEY (numPiece) REFERENCES piece(numpiece)  
    ON DELETE SET NULL ON UPDATE CASCADE);
```

- **ON DELETE SET NULL** : demande la mise à NULL des attributs constituant la clé étrangère qui font référence au tuple supprimé dans la table maître.
- **ON UPDATE CASCADE** : demande la modification des tuples dépendants, dans la table en cours de définition, quand le tuple contenant la clé primaire référencée est modifié dans la table maître.

Création d'une table: Contraints de domaine

- Les **contraintes de domaines** permettent de vérifier l'appartenance des valeurs d'un attribut à son domaine de définition.
- Les **contraintes de domaines** s'expriment par la clause :

CHECK <condition>

- **CHECK (condition)** : permet d'exprimer une condition qui doit exister entre plusieurs attributs de la ligne.
- *Exemple :*

Créer la table **Personnes** (**nump**, **nomp**, **sexp**) en exprimant une contrainte de domaine pour l'attribut **sexp** (F ou M)

```
CREATE TABLE Personnes (  
  nump NUMBER CONSTRAINT pk_personne primary key,  
  nomp VARCHAR(30),  
  Sexp CHAR ,  
  CONSTRAINT CK_sxsexe CHECK ( sexp IN ('M','F')  
);
```

Supprimer une table : **DROP TABLE**

- **Supprimer** une table revient à :
 - éliminer sa **structure** et toutes les **données** qu'elle contient.
 - Les **index** associés sont également supprimés.
 - les **lignes** de la table et la **définition** elle-même de la table sont détruites.
 - L'**espace** occupé par la table est libéré.
- **Syntaxe :**

DROP TABLE nom_table

- **Exemple :**

Supprimer la table **Article** (ref, nom, prix, datea):

DROP TABLE article ;

Modifier une table : ALTER TABLE

▪ Ajout de colonne : ADD

- On peut rajouter des attributs (ou des contraintes) oublié(e)s.
- Si la table contient déjà des données, les nouveaux attributs auront la valeur NULL pour les tuples présents.

▪ Syntaxe :

ALTER TABLE *nom_table* **ADD** ([*nom_colonne* **TYPE** [**CONTRAINTE**], ...])

▪ Exemple :

Ajouter l'attribut **email_valide** à la table **Personne** (**nump**, **nomp**, **semp**, **email**), en définissant une contrainte de validité ('o' ou 'n') sur cette colonne :

```
ALTER TABLE Personne
```

```
ADD(email_valide char(1)
```

```
CONSTRAINT personne_email_valide CHECK(email_valide IN ('o', 'n')));
```

Modifier une table : ALTER TABLE

▪ **Modification d'un type colonne : MODIFY**

- Il est possible de modifier le **type** d'un ou de plusieurs **attributs** (ou d'une **contrainte** si son nom est connu).
- **Syntaxe :**

ALTER TABLE *nom_table* **MODIFY** ([*nom_colonne* *type* [*contrainte*], ...])

▪ *Exemples :*

- Etendre la dimension de l'attribut **nomp** de la table **personne** à 50 caractères:
 - **ALTER TABLE** *personne* **MODIFY** (*nomp* **varchar**(50));
- On peut donner une contrainte de colonne dans la nouvelle définition de la colonne:

ALTER TABLE *personne* **MODIFY** (*sexe* **char**(1)

CONSTRAINT *personne_sexe_ck* **CHECK** (*sexe* **IN** ('m', 'f')));

Modifier une table : ALTER TABLE

▪ Supprimer une colonne : DROP COLUMN

- *Syntaxe :*

ALTER TABLE *table* **DROP COLUMN** *col*

- *Exemple:* Supprimer l'attribut **nomp** de la table personne:

ALTER TABLE *personne* **DROP COLUMN** *nomp*

- *Remarques:*

La colonne supprimée ne doit pas être **référéncée** par une **clé étrangère**.

▪ Renommer une colonne : RENAME COLUMN

- *Syntaxe :*

ALTER TABLE *table* **RENAME COLUMN** *ancien_nom* **TO** *nouveau_nom*;

- *Exemple :* Renommer l'attribut **email_valide** dans la table personne par **email_ok**

ALTER TABLE *personne* **RENAME COLUMN** *email_valide* **TO** *email_ok*;

Modifier une table : ALTER TABLE

▪ Ajouter, supprimer ou renommer une contrainte de table

Si des données existent dans la table au moment où la CI est ajoutée, toutes les lignes doivent vérifier la contrainte. Sinon, la contrainte n'est pas posée sur la table.

• Syntaxe :

○ Ajouter une contrainte :

```
ALTER TABLE nom_table ADD [CONSTRAINT nom_contrainte] contrainte
```

○ Supprimer une contrainte:

```
ALTER TABLE nom_table DROP CONSTRAINT nom_contrainte
```

○ Renommer une contrainte :

```
ALTER TABLE nom_table RENAME CONSTRAINT nom1 TO nom2
```

• Exemples :

- Dans la table EMP, supprimer la contrainte **nom_unique**.
- Dans la table EMP, ajouter une contrainte du salaire minimal (salaire=1000).

```
ALTER TABLE EMP
```

```
DROP CONSTRAINT nom_unique
```

```
ADD CONSTRAINT sal_min CHECK(SAL = 1000);
```

Modifier une table : ALTER TABLE

▪ Renommer une table : RENAME

- Syntaxe :

ALTER TABLE nom_table **RENAME TO** nouveau_nom;

- Exemple:

Renommer la table **Employé** (EMP) avec le nom **Salarier**

ALTER TABLE EMP **RENAME TO** Salarier;

- Exercise Application

Exercice

On considère le modèle Logique de données (MLD) suivant:

Client(code_cli, nom, prenom, ville)

Commande(num_cmd, date, #code_cli)

Produit(code_art, designation, qte_st, pu)

contenu_commande(#num_cmd, #code_art, qte_cmd)

facture(num_fact, date_fact, #code_cli)

Exercice d'initiation

Exercice

1. Créer la table client sans signaler que **code_cli** est une clé primaire ;
2. Modifier la table client pour que **code_cli** soit clé primaire ;
3. Ajouter le champ **adresse** à la table client
4. **Modifier** la taille de ce champ ;
5. **Supprimer** le champ adresse;
6. Créer la table **commande** en signalant que **num_cmd** est une clé primaire ;
7. Créer la table **Produit** en signalant que **code_art** est une clé primaire ;
8. Créer la table **contenu_commande** sans créer les relations;
9. Mettre en relation les tables Commande, Produit et Contenu_commande;
10. Ajouter la relation entre la table Client et la table Commande;
11. Créer la table **facture(num_fact,date_fact,#code_cli)**
12. **Supprimer** cette table de la base de données ;

MERCI